Making a pinball game for Playdate: Part 08, the entities and their components

Monday, May 26, 2025

Source: https://amano.games/devlog/making-a-pinball-game-for-the-playdate-part-08-the-entities-and-their-components

Welcome to this adventure, where I write about the process of our latest game, Devils on the Moon pinball.

I know I said the next post was going to be about the physics debugger, but I'm getting closer to finishing up a big refactor that had been on my TODO list for a long time. And I wanted to talk about it.

We are moving from an entity type/archetype to component-based behaviors.

Before

I had this structure for the game objects/entities of the game:

```
struct entity {
        enum entity_type type;
        u32 components;
        struct transform transform;
        struct rigid_body body;
        struct sensor sensor;
        struct sprite sprite;
        struct animator animator;
        ...
};
```

This was mirrored in the Level editor (Tiled), where I would build custom types that had the components needed for that entity type.



Then I would iterate over the entities, check which type they were, and perform some logic depending on the type.

Inside the function, I would assert that the entity had the components needed for that type to work, so something like:

```
// game.h
void game_upd(struct entity *entities, size count, f32 dt){
        for(size i = 0; i < count; ++i){
                struct entity *entity = entities[i];
                switch(entity->type):
                        case ENTITY_FLIPPER:
                                flipper_upd(entity, dt);
        }
}

// flipper.h

void flipper_init(struct entity *entity);

void flipper_upd(struct entity *entity, f32 dt) {
        assert(entity->components & COMPONENT_SPRITE);
        assert(entity->components & COMPONENT_RIGID_BODY);
        assert(entity->components & COMPONENT_TRANSFORM);
        struct sprite *sprite = &entity->sprite;
        // Update sprite
}
```

```
void flipper_drw(struct entity *entity);
```

This worked well for a while, but quickly I stopped updating every entity every frame, because some things didn't need to be updated unless they were on the screen or near a ball.

So I started caching the types of entities in the world and updating entities more based on their type, instead of going through the whole list every frame.

```
void flipper_sys_upd(struct world *world, f32 dt){
        struct entity *entities = world_query_entity_type(world, ENTITY_FLIPPER);
        for(size i = 0; i < arr_len(entities); ++i){
                flipper_upd(entities[i], dt);
        }
}
```

The more I did this, the more I realized a lot of the logic was at the component level, not really at the entity type level. So I started moving all the common logic for updating components to their own systems.

I used the same idea of making a DB of all the entities that had a specific component and using that for caching.

```
void sprite_sys_upd(struct world *world, f32 dt){
        struct entity *entities = world_query_component_type(world, COMPONENT_SPRITE);
        for(size i = 0; i < arr_len(entities); ++i){
                sprite_upd(&entities[i]->sprite, dt);
        }
}
```

The more I migrated the entity logic to the components, the more flexible the game code and level editor became.

I even created a new entity type called *Generic Entity* that we started using more and more. It would be a blank entity in Tiled by default, and then we could add components to it one by one.

At some point, I realized we could just get rid of the `entity_type` variable altogether. I was a little hesitant to change things, because we had a lot of entities already placed in the map, and migrating and checking them one by one seemed like a lot of work.

But then we stopped working on the game for 6 months to work on **Catchadiablos** (You can Pre-order it now!). When we came back, we had both forgotten 60% of the game's systems and functionalities. So it seemed like a good time to refactor this architectural design, and re-visit all the game functionality.

Now

It worked! It forced us to go through each entity, check what it was supposed to do, and ensure that after the refactor, it still behaved as intended.

If an entity needed a new behavior, I could create a new component and add it as a property to the Tiled entity without having to modify every entity of the same type.

This also allowed us to have optional components that wouldn't clutter the Tiled inspector, if an entity didn't need them, we simply wouldn't add them.

And if I need to implement logic for a specific type of entity archetype that requires a group of components to work properly, I can just create a new component and add it to any entity. Even if it's just a `bool`, its main purpose is to allow me to query for that component specifically and update the entity accordingly.

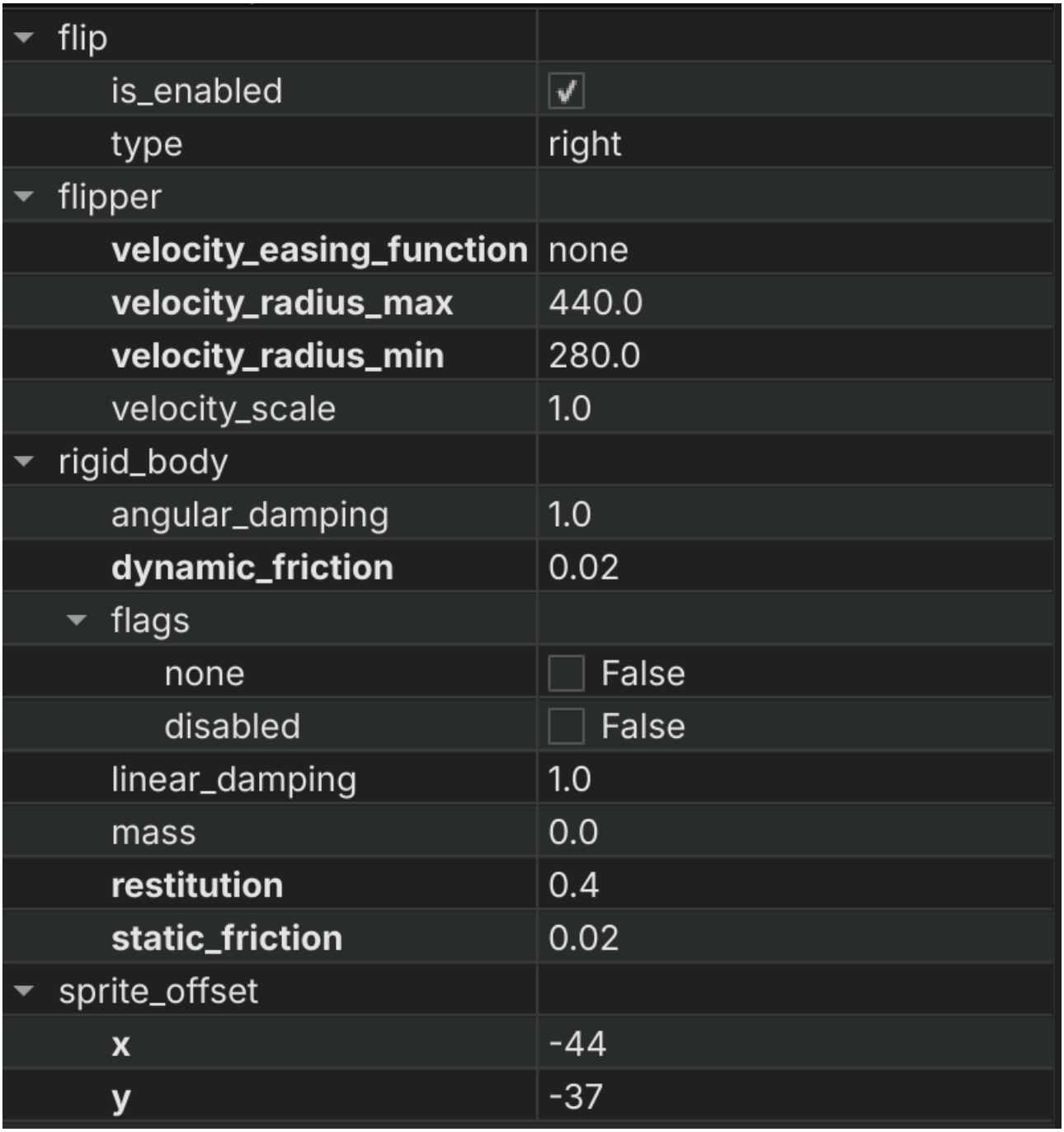
```
void flipper_upd(struct entity *entity, f32 dt) {
        assert(entity->components & COMPONENT_SPRITE);
        assert(entity->components & COMPONENT_RIGID_BODY);
        assert(entity->components & COMPONENT_TRANSFORM);
        assert(entity->components & COMPONENT_FLIPPER);
        struct flipper *flipper = &entity->flipper;

        // Do specific flipper logic
}

void flipper_sys_upd(struct world *world, f32 dt){
        struct entity *entities = world_query_component_type(world, COMPONENT_FLIPPER);
        for(size i = 0; i < arr_len(entities); ++i){
                flipper_upd(entities[i], dt);
        }
}
```

This way I keep the ability to have specific archetypes of entities but with the added flexibility for the more generic ones.

And this is how it currently looks in Tiled: we can reorder, rename, or remove any of the components and the game will keep working.



It doesn't look like much, but it helped us clean up the Tiled properties a lot. To convert from one type of entity to another, you just need to add or remove a component, instead of changing the entity type. Before, we needed to create a new entity type for each combination of components. Whereas now, we can remove and add components freely.

Another benefit is that now I can store the entity data in a more Data-Oriented way. So instead of having everything stored in the `entity` struct, we can have an array of components.

```
struct entity {
        u32 id;
        u32 components;
};
```

```
...

struct world {
        struct entity entities[100];
        struct sprite sprites[100];
};

void sprite_sys_upd(struct world, f32 dt){
        for(size i = 0; i < arr_len(world->sprites); ++i){
                sprite_upd(&sprites[i], dt);
        }
}
```

This seems obvious in hindsight, especially if you're familiar with Unity, where this is the norm. But when starting the project, it was easier to think about all the pieces separately and not try to generalize too much.

I don't know what awaits us in our next adventure, but see you there, when it happens.